字符串哈希进阶：双模、梅森质数与随机基数

> **来源**：Codex 讨论整理（XCPC 复习用） **适用**：判子串相等、字符串去重/分组、LCP/回文相关题；重点在「怎么选模数与基数、怎么防被卡」 **关联**：CCPC2025 网络赛 D 用单模 998244353；XCPC 数学手册「随机哈希判集合相等」同源思路

一、单模哈希与碰撞风险

字符串哈希通常用「多项式滚动哈希」：

```
h[i] = (h[i-1] * B + s[i]) % M;   // B 为基数，M 为大质数
```

子串 s[l, r) 的哈希（需要预处理幂数组 pw[i] = B^i mod M）：

```
hash(l, r) = (h[r] - h[l] * pw[r-l] % M + M) % M;
```

风险：

- 单模碰撞概率约 1/M，模数取 1e9+7 时约 10^-9；比较次数一多，碰撞概率不可忽视。
- 若基数固定且公开（131、13331），出题人可**针对性构造碰撞数据**（反哈希测试），固定模数同样可被构造。
- 结论：抗碰撞的两大手段是「模数规模够大」与「基数不可预测」。

二、双模哈希

做法：用两个不同的大质数 P1、P2 各做一遍哈希，存成一对 (h1, h2)；**两个都相等**才算字符串相同。

- 综合效果等价于模数 P1×P2 ≈ 10^18，单对碰撞概率降到约 10^-18，实战可视为不会撞。
- 常用组合：(1e9+7, 1e9+9)、(1e9+7, 998244353)。
- 代价：两套幂数组 + 两套前缀哈希，预处理 O(n)，判子串 O(1)，常数约翻倍。
- 注意：两个模数共用同一个基数 B；比较时绝不能只判其中一个。

三、梅森质数（2^k - 1）

典型值：2^31−1 = 2147483647、2^61−1 = 2305843009213693951。

优点：

- 因 2^k ≡ 1 (mod 2^k−1)，可用「折叠」快速取模：移位后把高位折回低位，无需通用除法。
- 2^61−1 ≈ 2.3×10^18，**一个梅森模数的抗碰撞水平 ≈ 两个 1e9 级质数相乘的双模**，所以 KACTL 等模板偏好「单哈希 + mod 2^61−1 + 随机基数」。

```cpp
using ull = unsigned long long;
const ull MOD = (1ull << 61) - 1;   // 2^61 - 1

ull mulmod(ull a, ull b) {
    __uint128_t t = (__uint128_t)a * b;
    t = (t >> 61) + (t & MOD);      // 折叠一次
    t = (t >> 61) + (t & MOD);      // 再折叠一次保证 < 2^61
    if (t >= MOD) t -= MOD;
    return (ull)t;
}
```

代价与坑：

- a、b 都接近 2^61，乘积达 2^122，必须用 **int128（GCC 扩展）或手工拆 64 位；**MSVC 没有** int128**。
- 折叠后要二次修正，代码比直接 `%` 容易写错。
- 结论：双模没必要换成两个梅森质数；梅森的价值是「用一个大模数顶替双模」。

四、随机基数（防反哈希的关键）

反哈希数据只能针对**已知的固定基数**预构造；运行期随机生成基数后，预构造数据即失效。

```cpp
ll getBase() {
    mt19937_64 rng(chrono::steady_clock::now().time_since_epoch().count());
    uniform_int_distribution<ll> dist(257, 1000000007 - 2);  // 避开 0、1、接近模数的值
    return dist(rng);
}
```

要点：

- 种子用 `chrono::steady_clock`，不要依赖 `random_device`（部分 OJ 实现是确定性的）。
- 全程序**只生成一次**，两个模数共用同一个 B；多组数据也复用（否则幂数组要配套重算）。
- 范围避开退化值：B=0（全串哈希 0）、B=1（哈希退化成字符和）、B 为模数倍数（模意义下为 0）。
- 模数本身可以是公开常数，安全不依赖模数保密。
- B 取到 1e9 量级时，h×B < 10^18，64 位不溢出，无需 __int128。

五、推荐模板：双模 + 随机基数

```cpp
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

const ll P1 = 1000000007, P2 = 1000000009;
const int MAXN = 1000005;   // 按题目最大串长调整

ll pw1[MAXN], pw2[MAXN];

ll getBase() {
```

```
    static mt19937_64 rng(chrono::steady_clock::now().time_since_epoch().count());
    uniform_int_distribution<ll> dist(257, P1 - 2);
    return dist(rng);
}

struct DoubleHash {
    ll B;
    vector<ll> h1, h2;
    DoubleHash(const string& s, ll base) : B(base) {
        int n = (int)s.size();
        pw1[0] = pw2[0] = 1;
        for (int i = 1; i <= n; i++) {
            pw1[i] = pw1[i-1] * B % P1;
            pw2[i] = pw2[i-1] * B % P2;
        }
        h1.assign(n + 1, 0); h2.assign(n + 1, 0);
        for (int i = 0; i < n; i++) {
            h1[i+1] = (h1[i] * B + s[i]) % P1;
            h2[i+1] = (h2[i] * B + s[i]) % P2;
        }
    }
    // 子串 s[l, r) 的双模哈希
    pair<ll, ll> get(int l, int r) const {
        ll a = (h1[r] - h1[l] * pw1[r-l] % P1 + P1) % P1;
        ll b = (h2[r] - h2[l] * pw2[r-l] % P2 + P2) % P2;
        return {a, b};
    }
};

int main() {
    string s = "example string";
    DoubleHash H(s, getBase());
    auto x = H.get(0, 7), y = H.get(8, 14);
    cout << (x == y ? "equal" : "different") << '\n';
}
```

若想用梅森风格的单哈希替代双模：

```
struct MersenneHash {          // KACTL 风格：单模 2^61-1 + 随机基数
    ull B;
    vector<ull> h, pw;
    MersenneHash(const string& s, ull base) : B(base) {
        int n = (int)s.size();
        h.resize(n + 1); pw.resize(n + 1);
        pw[0] = 1;
        for (int i = 1; i <= n; i++) pw[i] = mulmod(pw[i-1], B);
        for (int i = 0; i < n; i++) h[i+1] = (mulmod(h[i], B) + (ull)s[i]) % MOD;
    }
    ull get(int l, int r) const {   // 子串 s[l, r)
        ull x = (h[r] + MOD - mulmod(h[l], pw[r-l])) % MOD;
        return x;

    }
```

```
};
```

六、避坑清单

- 别用 int 存哈希：乘法必溢出，用 long long / unsigned long long。
- 子串做差后「+M 再 % M」防负数；漏加可能直接错。
- 双模必须两个都判；别只存一个哈希值就以为自己是双模。
- 基数固定 131/13331 的题解在线上可被卡；比赛模板默认走随机基数。
- 哈希是概率算法，若题目要求绝对正确，改用 KMP / 后缀数组 / SAM。
- 方案选择：省心选「双模 + 随机基数」（默认）；想要更短更强选「2^61−1 + 随机基数」（需 __int128）。